

Augmentation and Fusion: Multi-Feature Fusion-Based Self-Supervised Learning Approach for Traffic Tables

Xuan Zheng[✉], Xiuli Ma[✉], Lifu Xu[✉], Yanliang Jin[✉], and Chun Ke[✉]

Abstract—As modern networks face increasing demands for superior service and management, Encrypted Traffic Classification (ETC) technology has become increasingly crucial. Considering that traffic data is easy to collect but hard to label, self-supervised ETC methods have attracted more and more attention. Compared to popular methods based on traffic images and text, traffic tables are simple to construct and more suitable for the flow-packet structure. However, existing methods have two problems: (1) The lack of data augmentation methods for tables weakens the performance of self-supervised learning. (2) Most methods only focus on single feature and cannot make full use of distinct features of traffic tables, such as temporal feature. To solve these problems, we propose a multi-feature fusion method based self-supervised learning approach for traffic tables. A new data augmentation method called Random Subsets Selection (RSS) is introduced alongside an effective fusion approach. In this way, temporal features can be successfully extracted and concatenated with the latent representations of input traffic tables. Experimental results from two open datasets and one self-collected dataset have shown that on imbalanced datasets, our method can effectively solve ETC problems even with a small number of labeled data. Empirically, both classification performance and processing speed are improved. Specifically, compared to the state-of-the-art tabular self-supervised learning method, our method achieves the better classification results on all datasets while the processing speed increases by almost two times, from 1.83 tables per second to 3.76 tables per second.

Index Terms—Encrypted traffic classification, self-supervised learning, contrastive learning, tabular data.

I. INTRODUCTION

WITH the widespread use of smartphones and computers, the network has expanded dramatically and become more sophisticated. Although it brings huge convenience to our life, it also puts forward higher demands on network management. As one of the network management methods, Encrypted Traffic Classification (ETC) technology has demonstrated its importance in many aspects, such as malware

traffic identification, Quality of Service provisioning, and so on.

There is a lot of research on ETC methods. In the beginning, the port-based method [1] was very effective because of the simplicity of network. Since the dynamic port technology was widely applied, the performance of this method has declined a lot. After that, Deep Packet Inspection [2] was put forward and once achieved great results. This method heavily relies on packet payloads, and as a result its effectiveness has also suffered a great decrease in a more complex encrypted traffic environment. Hence, some research tries to use Machine Learning (ML) algorithms to do classification. However, ML-based methods depend heavily on human-engineered features, which will limit their generalizability.

Due to the development of Deep Learning (DL) algorithms and their powerful feature extraction capabilities, a lot of research has started applying them to the ETC problem. DL-based methods can be divided into two parts, data processing and classification algorithms. Considering that traffic data is not a common datatype and cannot be directly used by algorithms, data processing is necessary. Most methods first convert the raw traffic data into images or text, treating the ETC problem as an image classification problem or text classification problem. A corresponding classification algorithm is then applied to get expected results. Recently, research has indicated that traffic tables are more suitable for solving ETC problems [3]. For raw traffic data, there are two basic relationships: (1) multiple packets compose one flow, (2) the fields of different protocols consist of one packet. This structure is very similar to tabular data, and both relationships can be easily obtained in one specific table. For these reasons, raw traffic data are processed into traffic tables, whose rows represent packets and columns represent the fields of the packet header. A simple relationship between traffic tables and traffic data is shown in Fig. 1.

Traffic tables are important since they have three advantages. Firstly, compared to traffic images or text, one traffic table can simultaneously reflect the field-packet relationship (columns) and the packet-flow relationship (rows). Therefore, it can best represent the characteristics of raw traffic data. Secondly, traffic tables can avoid privacy invasion since only the packet header is used. Finally, temporal feature can be easily preserved by maintaining the order of packets. Based on the mentioned three merits, traffic tables are chosen to be the research interest.

Received 2 May 2024; revised 19 October 2024 and 6 February 2025; accepted 20 March 2025. Date of publication 28 March 2025; date of current version 9 June 2025. The associate editor coordinating the review of this article and approving it for publication was D. Pazaros. (Corresponding author: Xiuli Ma.)

Xuan Zheng, Xiuli Ma, Lifu Xu, and Yanliang Jin are with the School of Communication and Information Engineering, Shanghai University, Shanghai 200444, China (e-mail: zx19821217030@gmail.com; xlma@shu.edu.cn; xulifu@shu.edu.cn; wuhaide@shu.edu.cn).

Chun Ke is with H3C, Shanghai 200082, China (e-mail: ke.chun@h3c.com).

Digital Object Identifier 10.1109/TNSM.2025.3554824

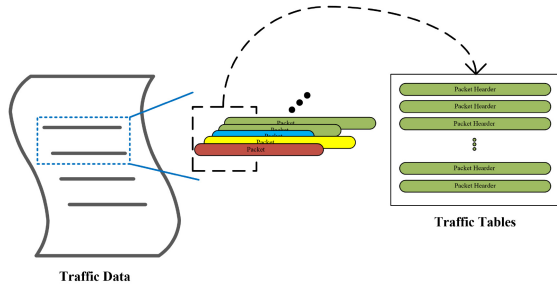


Fig. 1. The relationship between traffic data and traffic tables.

Considering that it is easy to collect traffic data but hard to label them, many studies try to leverage abundant unlabeled data to mitigate the dependence on labeled data. However, traffic tables lack data augmentation methods, which will influence the performance. Besides, there are some special features existing in traffic tables, such as the temporal feature. How to make full use of these feature is also an urgent problem.

To solve these problems, we propose a multi-feature fusion based self-supervised learning method for traffic tables. A new pretext task is proposed to learn latent representations of input traffic tables without labels, reducing the dependence on labeled data. It contains a reconstruction part and a contrastive part. Considering that contrastive learning requires various data augmentation methods to achieve great performance, a new traffic tables augmentation method called Random Subsets Selection (RSS) is proposed to further augment traffic tables. Specifically, input traffic tables are first divided into several sub-tables, and then some of them are randomly selected to aggregate the latent representations. Secondly, the random selection is applied again for the rest of the sub-tables. However, this time these sub-tables will be added noise and used to aggregate the corrupted latent representations. After that, both latent representations are used for the reconstruction and the contrast tasks. Compared to common methods, our method can effectively find the informative sub-tables and give them bigger weights. Moreover, an effective temporal features extraction and fusion method is also proposed. Specifically, after obtaining the reconstructed input, we use mean pooling on each column and concatenate it to the latent representations.

From the experimental results, our method performs the best on two open datasets, ISCX VPN-nonVPN and ISCX Tor-nonTor datasets, and ranks the second on the self-collected dataset SHU-ET. The best representations of input traffic tables can be learned and clustered through pretraining. In this way, great performance can be obtained with even a small number of labeled samples. With addressing the mentioned two problems, our method surpasses the state-of-the-art self-supervised tabular learning methods on both classification performance and processing speed.

The main contribution can be summarized as follows:

- 1) A new tabular data augmentation method RSS is proposed. It addresses the limitation of single data augmentation approach for traffic tables in self-supervised or contrastive learning.

- 2) An effective temporal feature extraction and fusion method is proposed. It can improve the model's performance without introducing additional neural network modules.
- 3) A new pretext task for tabular-based self-supervised learning is proposed, combining reconstruction and contrastive tasks together. Compared to common methods, it improves both classification performance and processing speed.

The rest of this article is organized as follows: Section II will introduce the related work. Section III will discuss the used datasets and give the problem formulation. Section IV will elaborate our methodology in detail, including the data processing method and the network structure. Comprehensive experiments have been conducted to evaluate the performance of the proposed method, which can be found in Section V. What's more, we do ablation study to further analyze the effectiveness of proposed strategies, presented in Section VI. At last, in Section VII, we will summarize our work and come to a conclusion.

II. RELATED WORK

ETC aims to classify traffic data according to their applications or services. In this section, we will introduce the related work in detail, and the overall differences of them are presented in Table I.

A. Classification Methods Based on Statistical Features

Most machine learning algorithms first manually extract statistical features of traffic data, and then use these statistical features to do classification. Auld et al. [4] first comprehensively extracted 249 statistical features of network traffic and listed a features table. A Bayesian neural network was trained through these statistical features and achieved great classification results [5]. Shen et al. [6] proposed to only use packet length information to obtain a fine-grained webpage fingerprint. Recently, Xu et al. [24] proposed to construct a traffic path with a session packet length sequence. It was computed as a kind of distinct feature and then trained regular ML models. Although machine learning methods have achieved great results, these methods heavily depend on human-engineered features. Different encrypted techniques will change these features, which will limit these models' generalization performance.

B. Classification Methods Based on Traffic Images

Since DL algorithms have powerful feature extraction capabilities, they have been widely applied to research ETC. Considering that there are a lot of powerful methods existing in Computer Vision (CV), a common strategy is constructing traffic images through traffic packets or flows. Wang et al. [7] firstly proposed to generate traffic images by traffic packets. After obtaining traffic images, image classification methods were directly used to classify them. Besides, Wang et al. [8] also compared the 1dCNN algorithm and 2dCNN algorithm and found that 1dCNN was better for traffic images. Shapira and Shavitt [25] proposed a different way to construct

TABLE I
SUMMARY OF RELATED WORK

Input Datatype	Learning Type	Encrypted Traffic Classification	
		Name (Year)	
Statistical Features	Supervised	Moore et al. [4], [5] (2005, 2007), Shen et al. [6] (2020), Xu et al. [5] (2022)	
Traffic Images	Supervised	Wang et al. [7], [8] (2017), Shapira et al. [9] (2021)	
	Self or Semi-supervised	Lin et al. [10] (2022), Iliyasu et al. [11] (2019), Aouedi et al. [12] (2020), Zhao et al. [13] (2022)	
Traffic Text	Supervised	Lotfollahi et al. [14] (2020)	
	Self or Semi-supervised	Lin et al. [15] (2022), He et al. [16] (2020), Wei et al. [17] (2021)	
Integrated datatype	Supervised	Zuo et al. [18] (2018), Lin et al. [19] (2021), Zheng et al. [20] (2019)	
tabular	Self-supervised	Zheng et al. [3] (2023), Yoo, Zhang et al [21] (2020), Ucar et al. [22] (2021), Bahri. et al. [23] (2021)	

traffic images called FlowPic. It used packet lengths as values and arrival time as parameters. Since traffic image constructing strategies were similar, some people turned to figure out how to apply self-supervised and semi-supervised algorithms to them. Lin et al. [10] proposed to generate pseudo-labeled data to reduce the dependency on labeled data. It contained an auxiliary network and autoencoder, which was called Sauce. Since DCGAN was very popular in the CV field, Iliyasu and Deng [11] applied it to generate pseudo images. These pseudo images helped the model learn features from unlabeled data and realized semi-supervised learning. Aouedi et al. [12] develop a semi-supervised model by applying a stacked sparse autoencoder. Zhao et al. [13] proposed to apply mean teacher structure to solve semi-supervised learning problems.

C. Classification Methods Based on Traffic Vectors or Text

Since traffic packets are one-dimensional vectors, a lot of studies try to use Natural Language Processing (NLP) methods to solve ETC problems. Lotfollahi et al. [14] proposed Deep Packet which directly fed traffic payloads to both SAE and 1dCNN to extract comprehensive features. Also, there are a lot of efforts to apply popular NLP self-supervised or semi-supervised learning methods to traffic data. Since BERT has achieved great success in the NLP domain, Lin et al. [15] and He et al. [16] used the same idea and proposed ET-BERT-based and PERT-based methods, respectively. These methods viewed traffic data as text and constructed a special dictionary. Then directly use BERT algorithm to pretrain a model on a large amount of unlabeled data, and fine-tune it with few labeled data to fit the downstream classification task. Wei et al. [17] was a little different. It first focused on processing and extracting local short-term characteristics of traffic data, and then also used BERT to do the other jobs.

D. Classification Methods Based on Integrated Algorithms

Since methods with a single feature were not enough to achieve better performance, some research tried to extract multiple features from both traffic images and text. Zuo et al. [18] combined CNN and LSTM to extract the packet features from traffic images and temporal features from the packet payload. Lin et al. [19] also tried to extract these two features, but used bidirectional LSTM instead of LSTM. What's more, Zeng et al. [20] applied CNN, LSTM, and SAE to get more complicated traffic features. Although these Integrated methods can improve results to some extent, the networks also become larger and more complicated. Besides,

almost all these methods used encrypted payload of packets which is greatly affected by the environment and user habits. As a result, their generalization performance may be also constrained.

E. Self-Supervised Learning Methods for Tabular Data

Our previous work [3] proposed to construct traffic tables from packet headers. Traffic tables are suitable to solve ETC problems since traffic data are very similar to tabular data. Supervised tabular classification methods heavily depend on the labeled data. However, for traffic data, they are easy to be collected by hard to be labeled. As a solution, self-supervised tabular classification methods were studied. Yoon et al. [21] proposed that adding swap noise can help the model learn better latent representations from abundant unlabeled data and improve the classification results. Rezaei and Liu [22] first divided the original tabular data into subsets and then learned the latent representations from the subsets instead of the whole input tables. Fu et al. [23] proposed a contrastive learning approach by simply adding noise to original tables.

However, the lack of data augmentation methods for tabular data influences the final performance. Besides, how to make full use of the distinct features existing in traffic tables is also a problem. As a result, a new approach should be found to solve these problems.

III. DATASETS AND PROBLEM FORMULATION

A. Datasets

To comprehensively evaluate the performance, three different datasets are used during experimental stage. ISCX VPN-nonVPN dataset is an open dataset and released by Canadian Institute for Cybersecurity. It is widely used as the benchmark dataset in ETC field with a size of 28GB. Specifically, all traffic data in it are captured by Wireshark and tcpdump. Meanwhile, researchers use the UDP model of the OpenVPN to generate encrypted VPN traffic data. Referred to Wang's method [7], 12 classes of traffic data are used in our research, which consists of 6 normal traffic service types and their corresponding VPN encrypted version. Table II shows the specific traffic classes and contents.

The other used open dataset is ISCX Tor-nonTor dataset, which is also released by Canadian Institute for Cybersecurity. Its size is 22GB and shares some features of ISCX VPN-nonVPN dataset. The main difference is that pairs of traffic data are simultaneously captured at the gateway and workstation, instead of generating extra VPN data. The traffic data

TABLE II
ISCX VPN-nonVPN DATASET

Traffic Class	Contents
(nonVPN + VPN) Email	SMTPS, POP3S, IMAPS
(nonVPN + VPN) Chat	ICQ, AIM, Skype, Facebook, Hangouts
(nonVPN + VPN) Streaming	Vimeo, YouTube
(nonVPN + VPN) File Transfer	Skype, FTPS, SFTP
(nonVPN + VPN) VoIP	Facebook, Skype, Hangouts voice calls
(nonVPN + VPN) P2P	uTorrent, Transmission (Bittorrent)

TABLE III
ISCX TOR-nonTOR DATASET

Traffic Class	Contents
(nonTor + Tor) Email	SMTPS, POP3S, IMAPS
(nonTor + Tor) Chat	ICQ, AIM, Skype, Facebook, Hangouts
(nonTor + Tor) Streaming	Vimeo, YouTube
(nonTor + Tor) File Transfer	Skype, FTPS, SFTP
(nonTor + Tor) VoIP	Facebook, Skype, Hangouts voice calls
(nonTor + Tor) P2P	uTorrent, Transmission (Bittorrent)

TABLE IV
SHU-ET DATASET

VPN Types	Applications
ExpressVPN	Facebook, Telegram, Twitter, YouTube
SSRVPN	Facebook, Telegram, Twitter, YouTube
V2Ray	Facebook, Telegram, Twitter, YouTube

captured at the workstation are labeled as regular, while the other data captured at the gateway are labeled as Tor. Like the ISCX VPN-nonVPN, this dataset also consists of 12 different traffic service types. Table III shows the detail of ISCX Tor-nonTor dataset.

The third dataset is a self-collected dataset, which is called SHU-ET. We apply three different typical VPN techniques (ExpressVPN, SSRVPN and V2Ray) to encrypt four applications (Facebook, Telegram, Twitter, YouTube), which is shown in Table IV. Therefore, it also contains 12 different classes and its size is 9.73 GB. Compared to the previous two open datasets, there are two main differences. One is that the data collected in SHU-ET is mobile traffic, which means they are generated on mobile devices. The other is that labels of traffic are changed from network services to specific applications. Furthermore, three types of VPN stand for different encryption techniques. The main purpose of using SHU-ET is to test the model's performance on classifying applications and encryption techniques. Accordingly, SHU-ET can be used as a supplement to evaluate the model's generalization capability.

Table V shows overall differences of three datasets.

B. Problem Formulation

In this subsection, we introduce the general formulation of tabular-based self-supervised ETC. It contains two tasks, one is unlabeled self-supervised learning and the other is labeled traffic classification. Suppose we have a dataset consisting of a set of traffic classes C . For a certain traffic C_i , it contains a number of traffic flows F . Furthermore, for a specific flow F_j , it is made up by several traffic packets P . An ETC task can be either classifying traffic flows or traffic packets, which

TABLE V
THE OVERALL DIFFERENCE OF THREE DATASETS

Dataset	Size	Classes Number	Classification Task	Samples
ISCX VPN-nonVPN	28GB	12	Services	193346
ISCX Tor-nonTor	22GB	12	Services	211980
SHU-ET	9.73GB	12	Applications	78762

can be called flow-level ETC task and packet-level ETC task, respectively. Our research tries to solve the flow-level ETC task since traffic flows usually contain more features. What's more, if a certain flow is correctly classified, all packets in it can also be identified.

Considering that traffic flows cannot be directly used by deep learning models, data processing is necessary. Unlike general methods constructing traffic images or text, we convert flows into traffic tables. Specifically, for a certain flow F_j , we continuously split it into a table T_k every n packets. Besides, packet payloads are dropped and only packet headers without IP addresses and ports information are used in order to protect privacies and avoid overfitting. For each packet header, m fields are used to constitute columns of traffic tables. Therefore, the dataset can be described as a small labeled dataset $D_l = \{T_i, y_i\}_{i=1}^{N_l}$ and a large unlabeled dataset $D_u = \{T_i\}_{i=1}^{N_l+N_u}$, where $N_l \ll N_u$ and $T_i \in \mathbb{R}^{n \times m}$. The model is pretrained with large unlabeled dataset D_u to learn best representations of input traffic tables. For downstream classification task, the small labeled dataset D_l is used to realize accurate classification.

1) *Self-Supervised Learning*: Self-supervised learning aims to learn useful information and best representations of input traffic tables. Learned representations can be utilized by downstream tasks and effectively reduce the demand for labeled data. Normally, this progress is completed by setting various self-supervised/pretext tasks to train a pretext model. Specifically, an Encoder e will code the input traffic table T_i to an informative representation z_i . The representation z_i will be used to solve the pretext task by minimizing the self-supervised loss l_{ss} with pseudo-label y_s . For example, the widely used pretext task is a reconstruction strategy, which reconstructs the input table T_i from its corrupted version. In this circumstance, y_s is the original input T_s and l_{ss} is the squared difference between the reconstruction output X' as follows,

$$\min \mathbb{E}_{(T_s, y_s) \sim p_{T, Y}} [l_{ss}(y_s, e(T_s))] \quad (1)$$

where $p_{T, Y}$ is the distribution used to generate pseudo-labeled samples (T_s, y_s) and train model e . For downstream task, the Encoder can be trained jointly with the classification model. The pretext task is vital to final performance, thus it should be carefully designed. In CV and NLP domain, there already have massive pretext tasks benefited from the plentiful data augmentation methods. However, in tabular domain, the pretext task is less diverse and dominated by the mentioned reconstruction strategy. As a consequence, it is necessary to design a new powerful pretext model.

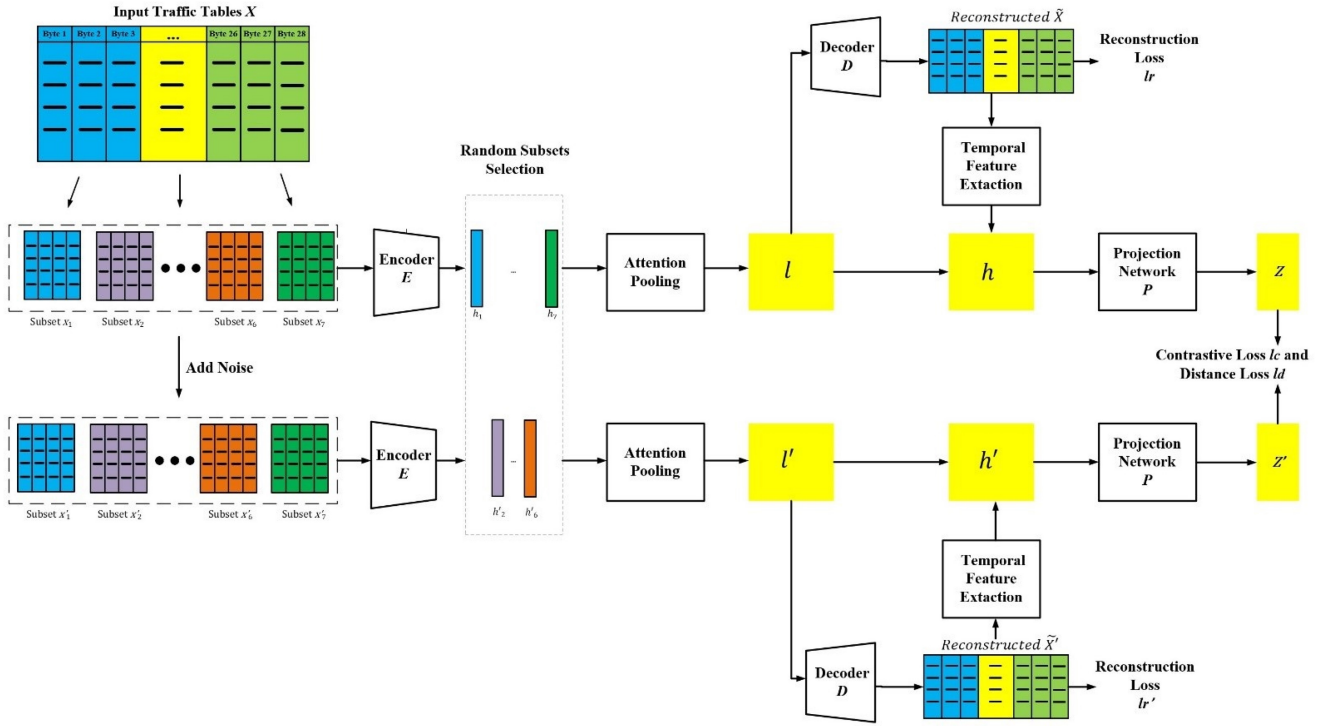


Fig. 2. The framework of the training stage.

2) *Downstream Traffic Classification*: For downstream task, the Encoder e is trained jointly with a classification model c using only the small labeled dataset D_l . A predicted label $\tilde{y}$ will be outputted and used to minimize the training loss with the real label y .

Overall, the main task is to design an effective pretext model, which can learn the most informative representation of the input traffic tables. For downstream classification task, great results are expected to be obtained with a labeled dataset, which is expected to be as small as possible.

IV. METHODOLOGY

A. Traffic Tables Generation

The generation of traffic tables contains 6 steps similar to our previous work [3].

Step 1: Filter irrelevant packets, such as DNS, ARP, and SNMP packets.

Step 2: Exact IP headers and TCP/UDP headers.

Step 3: Delete 4-byte source IP address and 4-byte destination IP address in IP headers.

Step 4: Delete 2-byte source port and 2-byte destination port in TCP/UDP headers.

Step 5: Padding UDP headers with zeros to match the length of TCP headers.

Step 6: Forming a table by sequentially selecting 14 packets and converting data from hexadecimal to decimal.

The packet header consists of fixed fields according to the protocol. This structure is the same as tabular data and can be directly used to generate traffic tables. Since many studies have proved that IP address information will cause the overfitting problem, we delete 4-byte source IP address and

4-byte destination IP address. As a result, the used IP-header length is 12 bytes. What's more, considering the widespread adoption of dynamic port technology, we also delete 4-byte port fields. Thus, the used TCP-header length is 16 bytes and the used UDP-header length is 8 bytes. For the sake of uniform structure, UDP headers are padded with zeros to match the length of TCP headers. At last, the total length of the used header is 28 bytes (12 bytes IP header and 16 bytes TCP/UDP headers). Consequently, the number of columns for each traffic table is 28. Meanwhile, the order of packets is unchanged since it represents the temporal feature of traffic data. Although the number of columns is easy to be confirmed, it is challengeable to decide how many packets should be used to construct one traffic table. Through experimental results, the optimal row count for one traffic table is found to be 14.

B. Network Structure

In this article, an extensible self-supervised learning method is proposed to solve the ETC problem. It consists of a reconstruction part and a contrastive learning part. The reconstruction part is responsible for learning the latent representations of input traffic tables. The contrastive learning part maximizes the agreement of differently augmented tables, which is very similar to SimCLR [26]. For the characteristics of traffic tables, certain improvements have been made. At the training stage, large amounts of unlabeled data are used to learn their corresponding latent representations. The framework is displayed in Fig. 2.

1) *Reconstruction*: First, a reconstruction method is designed to do representative learning. To obtain the

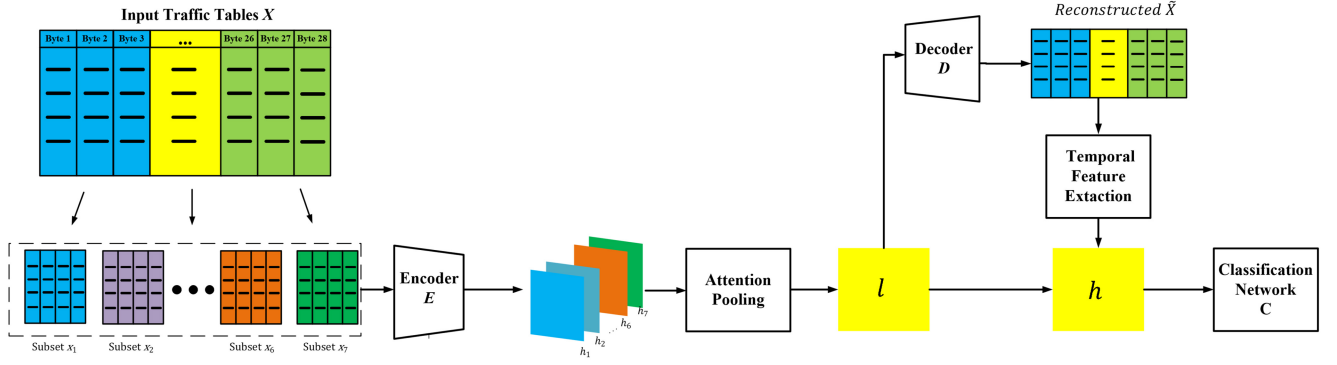


Fig. 3. The framework of the test stage.

corresponding latent representation, the traffic table X is sent to an encoder E . In contrast to conventional methods [21], [27], we do not send the whole original traffic tables to the encoder. Instead, X will be chopped into N small sub-tables, denoted as x_i . The reason is that different fields are not equally informative. In other words, some fields have a significant impact on classification while others are not. It is helpful to obtain better latent representations by giving sub-tables different weights. After sending each x_i to the encoder, their corresponding latent representations are obtained, denoted as h_i . After attention pooling, h_i will be given different weights, and then be used to aggregate the input representation l . After that, a decoder D is used to reconstruct the original input table X from l .

2) *Temporal Feature Extraction*: One of the most distinct features within traffic tables is the temporal feature. In traffic tables, rows represent the packets and their positions indicate the order of their arrivals. As a result, we can use columns to extract temporal features. Considering that the count of columns is fixed in each traffic table, it can be viewed as a time series. For each row (packet), we compress all columns' information to one dimension, which is the same as pooling. We use mean values to represent the columns' information. The reason is that for each packet, the minimum value of the header fields is always 0 and the maximum value is also very close. Thus, the mean value can best represent the differences between packets. What's more, we extract the temporal features from the reconstructed $\tilde{X}$ and $\tilde{X}'$. After the reconstruction, we can treat $\tilde{X}$ and $\tilde{X}'$ as the original traffic tables, to some extent. Since we add swap noise to the subsets, the original temporal distribution is changed, which makes the extracted temporal features a little different. However, for the same traffic table, the temporal feature should be the same and it can be optimized by contrastive loss. In a word, we do mean pooling on reconstructed $\tilde{X}$ and $\tilde{X}'$. After the reconstruction, we can use $\tilde{X}$ and $\tilde{X}'$ to extract temporal features. In a word, the dimension of temporal features is 14×1 , and then they are concatenated to the aggregated representations l and l' , generating the fusion representations h and h' .

3) *Contrastive Learning*: The contrastive learning method is very similar to SimCLR [1]. It has already been found that the combination of multiple data augmentation operations is crucial in defining contrastive prediction tasks. Consequently,

two data augmentation methods are applied to traffic tables, and one is adding noise which is widely used. The other is RSS algorithm, which is proposed to solve the problem that traffic tables lack data augmentation methods.

For the noise strategy, noises are added to sub-tables instead of the input table. If directly adding noise to the whole traffic table, all features will be treated equally [28]. However, fields are not equally informative to the classification. Therefore, treating them equally will cause unintended results. As a solution, adding noise to sub-tables can help model find the important fields.

The RSS algorithm is proposed to do further augmentation. Each original traffic table X is divided into N (7 in our work) sub-tables. First randomly choose n_1 (from 1 to $N - 1$) sub-tables of X . These sub-tables will be sent to the attention pooling module and used to aggregate the latent representation h . Then in the rest ($N - n_1$) sub-tables, randomly choose n_2 (from 1 to $N - n_1$) sub-tables, and their corresponding corrupted subsets are used to aggregate the latent representation h' .

It is similar to the combination of chopping and noising in images. Then a learnable nonlinear projection network is used to project h and h' to z and z' , respectively. Finally, the contrastive loss is calculated between z and z' .

4) *Test Stage*: At the test stage, the trained encoder and attention pooling module are transferred to the downstream classification test. The network structure is displayed in Fig. 3.

The input traffic tables will be divided into subsets and then sent to the trained encoder and attention pooling module. This time all the N sub-tables will be used to aggregate the latent representation. The encoder and attention pooling module are frozen and the parameters will not be upgraded. After that, these representations will be used to train a classification network C (logistic regression network), and a test set will be used to evaluate performance. Besides, both the entire labeled dataset and only a portion of it can be used to train the network.

C. Loss Function

There are three loss terms in this method, which are reconstruction loss l_r , contrastive loss l_c and distance loss l_d .

1) *Reconstruction Loss*: Given an input feature, denoted by X , we can reconstruct the entire feature $\tilde{X}$. The reconstruction

loss of the original input is equal to the mean squared error using $(X, \tilde{X})$ pair. Overall reconstruction loss is given below:

$$l_{ro} = \frac{1}{N} \sum_{i=1}^N \left(X^{(i)}, \tilde{X}^{(i)} \right)^2 \quad (2)$$

where N is the number of batch sizes.

Besides, since the entire feature X can also be reconstructed by corrupted sub-tables, denoted by X' . The reconstruction loss can also be calculated by (X, X') pair, like given below:

$$l_{rc} = \frac{1}{N} \sum_{i=1}^N \left(X^{(i)}, \tilde{X}'^{(i)} \right)^2 \quad (3)$$

At last, the total reconstruction loss is a weighted sum of l_{ro} and l_{rc} , which is given below:

$$l_r = \alpha l_{ro} + (1 - \alpha) l_{rc} \quad (4)$$

2) *Contrastive Loss*: The contrastive loss is similar to SimCLR. For n examples, after data augmentation, there will be $2n$ data points. Given a positive pair, the other $2(n-1)$ augmented examples are treated as negative. Then the loss function for a positive pair of examples (i, j) is given below:

$$l_c = -\log \frac{\exp\left(\frac{\text{sim}(z_i, z_j)}{\tau}\right)}{\sum_{k=1}^{2n} 1_{[k \neq i]} \exp\left(\frac{\text{sim}(z_i, z_k)}{\tau}\right)} \quad (5)$$

where $1_{[k \neq i]} \in (0, 1)$ is an indicator function evaluating to 1 if $k \neq i$ and τ denotes a temperature parameter.

3) *Distance Loss*: Mean-Squared Error (MSE) loss is used to compute the distance loss for projections of fusion representations z and z' , the overall MSE loss is given below:

$$l_d = \frac{1}{N} \sum_{i=1}^N (z - z')^2 \quad (6)$$

where N stands for the batch size.

Final the total loss function l_{total} is a weighted sum of l_r , l_c and l_d , which is given below:

$$l_{total} = l_r + \beta l_c + \gamma l_d \quad (7)$$

where l_r is computed by equation (3), containing a weight parameter α .

V. EVALUATION

In this section, we first introduce the datasets and experimental setup and then compare our method with the state-of-the-art methods to evaluate the performance.

A. Dataset

All three datasets mentioned in Section III will be used to evaluate the proposed method. First, the dataset will be divided into the training set and the test set. The ratio of these two sets will be fixed to 4:1. Besides, the sample proportion of each class is also strictly preserved. For the pretraining task, all training data without labels are used to help the model learn their best representations. When dealing with the downstream classification task, the labeled training data will be used. To

evaluate the model's reliance on labeled data, we will change the size of labeled training data while keep the test data as the same.

B. Experiment Settings

1) *Experimental Environment*: The experiments are conducted on a server with AMD Ryzen R5-5600@3.6Ghz, 32GB RAM, and an NVIDIA GeForce RTX3070 GPU. The implementation of our method is based on Python 3.7.

2) *Evaluation Metrics*: Four common metrics are used to evaluate our method, denoted Accuracy, Recall, Precision, and F1-Score.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (8)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (9)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (10)$$

$$\text{F1-Score} = \frac{2 \times \text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}} \quad (11)$$

where TP , FP , TN , and FN represent true Positive, False Positive, True Negative, and False Negative, respectively.

C. Optimal Row Count

To find the best construction method for traffic tables, optimal row count should be taken into consideration. In this experiment, only ISCX VPN-nonVPN dataset is used. For the downstream task, different labeled data proportions are used to comprehensively evaluate the influence of various row counts. Fig. 4 shows the relationship between the row count and the classification performance. As the number of rows in a single table increases, the model consistently exhibits a trend of initial improvement followed by decline. It is apparent that either excessively high or excessively low row count will weaken the model's performance. For traffic tables with high row count, the number of samples will be relatively small. For example, if one flow contains 56 packets and the row count for a traffic table is chosen to be 28, 2 samples can be generated. However, if the row count is selected to be 56, the generated sample will be only 1. Besides, the feature pattern of a certain traffic table will be too complicated as well. As a result, it makes encoder harder to learn latent representations. However, sufficient informative features may not be included in one traffic tables with a low row count, which also causes a decline in performance. From Fig. 4, when we use 14 and 21 packets to construct traffic tables, the model can achieve better performance. In order to obtain the best performance with a small number of labeled samples, optimal row count is selected to 14 and used for all the following experiments.

D. Performance on Three Different Datasets

The overall performance on three different datasets is given in this subsection. During the pretraining stage, the model is designed to learn best latent representations of traffic tables, which can be reflected by a great cluster performance. Thus, we use PCA and t-SNE plots to analyze this performance on

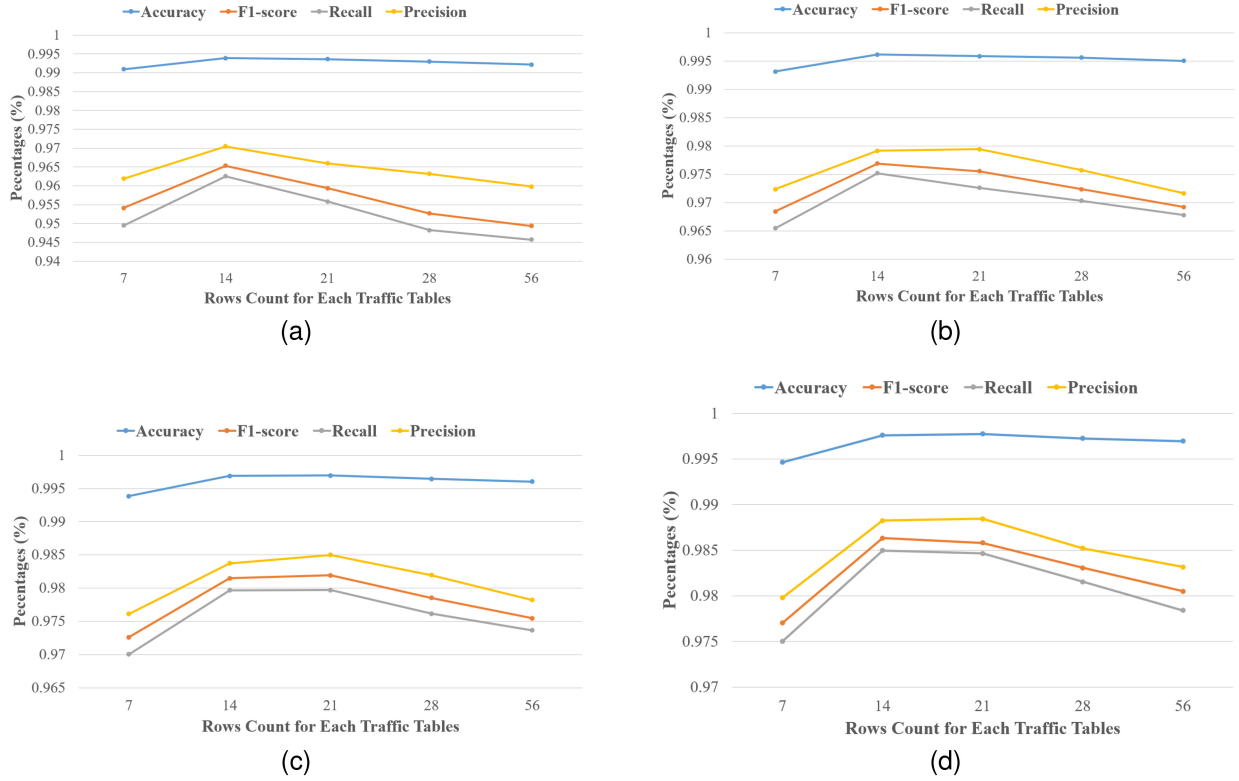


Fig. 4. The relationship between different row count and the model's performance. (a), (b), (c) and (d) show the performance trends with different labeled data ratios used for training the downstream classification network, which is 10%, 30%, 50% and 100%, respectively.

TABLE VI
THE DISTRIBUTION OF SAMPLES IN THREE DATASETS

ISCX VPN-nonVPN			ISCX Tor-nonTor			SHU-ET		
Class	Samples	Percentages(%)	Class	Samples	Percentages(%)	Class	Samples	Percentages(%)
Chat	2149	1.11	Chat	151	0.07	e-Facebook	11185	14.20
Email	521	0.27	Email	8109	3.83	e-Telegram	309	0.39
File-Transfer	46015	23.80	File-Transfer	10219	4.82	e-Twitter	14065	17.86
P2P	7749	4.01	P2P	7749	3.66	e-YouTube	5695	7.23
Streaming	2951	1.53	Streaming	18950	8.94	s-Facebook	3713	4.71
VoIP	58913	30.47	Tor-Chat	152	0.07	s-Telegram	2234	2.84
VPN-Chat	543	0.28	Tor-Email	5556	2.62	s-Twitter	9024	11.46
VPN-Email	1347	0.70	Tor-File-Transfer	8847	4.17	s-YouTube	6965	8.84
VPN-File-Transfer	2294	1.19	Tor-P2P	103497	48.82	v-Facebook	4519	5.74
VPN-P2P	30148	15.59	Tor-Streaming	19313	9.11	v-Telegram	1712	2.17
VPN-Streaming	14881	7.70	Tor-VoIP	15558	7.34	v-Twitter	9696	12.31
VPN-VoIP	25835	13.36	VoIP	13879	6.55	v-YouTube	9645	12.25

The **red bold** font represents the class with the largest proportion.

the three datasets, as shown in Fig. 5. From PCA plots, it can be found that original data distributions are chaos. Through pretraining, data distributions achieve a great improvement. As designed, most positive samples are pulled together and negative samples are pushed away, which is represented in t-SNE plots. Therefore, even a small number of labeled data can be used to train a great classification network.

What's more, we do further analysis on datasets and Table VI provides a comprehensive data distribution for each dataset. It is evident that traffic datasets, particularly the ISCX VPN-NonVPN and ISCX Tor-NonTor datasets, suffer from severe class imbalance. In the ISCX VPN-NonVPN dataset, VoIP samples dominate at 30.47%, followed by File-Transfer, VPN-P2P, and VPN-VoIP, collectively making up 83.22% of

the dataset, while the remaining 8 classes account for only 16.78%. In the ISCX Tor-NonTor dataset, Tor-P2P samples constitute nearly 50%, vastly outnumbering Chat and Tor-Chat. The SHU-ET dataset is relatively balanced, with the largest sample difference being less than 50 times, and only the e-Telegram class under 1%. This imbalance stems from the varying traffic volumes generated by different services, such as Streaming versus Chat. Since real-world data distribution is unknown without labels, the imbalance is retained. As a result, this imbalance will lead to an overall inflated classification accuracy, which can be reflected in the confusion matrices, as shown in Fig. 6. In this subsection we want to analyze the model's performance on each class in a certain dataset, so only 100% labeled data trained results are given. In ISCX

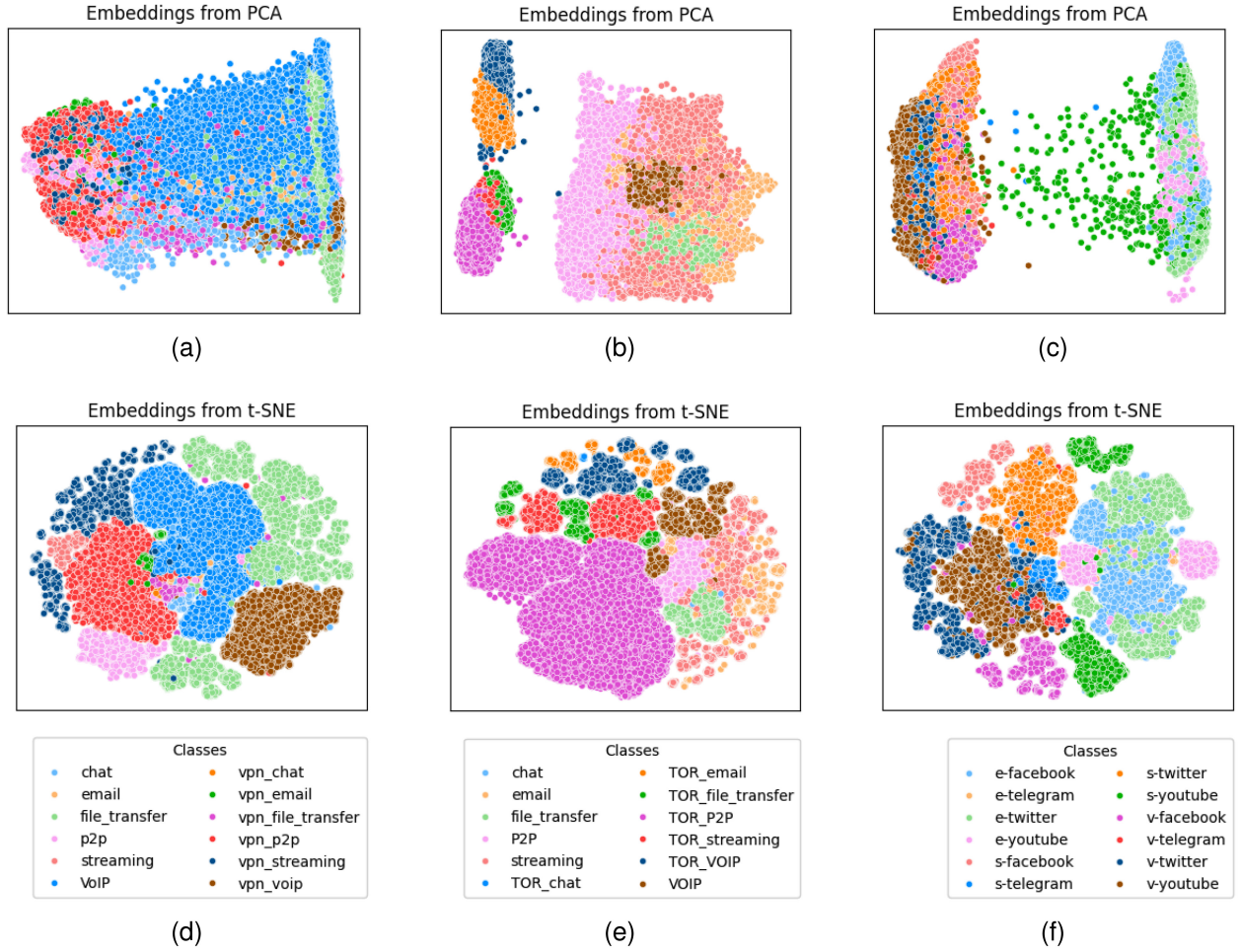


Fig. 5. PCA and t-SNE plots of three datasets. From left to right, (a), (b) and (c) represent PCA plots of ISCX VPN-nonVPN, ISCX Tor-nonTor and SHU-ET dataset, respectively. Figures below them are corresponding t-SNE plots and class labels. Specifically, (d), (e) and (f) are t-SNE of ISCX VPN-nonVPN, ISCX Tor-nonTor and SHU-ET dataset, respectively.

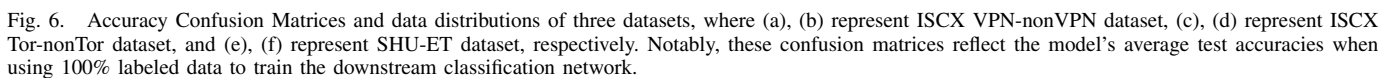
VPN-nonVPN and Tor-nonTor datasets, accuracies of Chat and Email classes are worse than other classes.

The reason is that these two services are similar to each other, which mainly contains text data and generates only a small number of traffic, like shown in Fig. 6(b) and Fig. 6(d). Thus, it's hard for the model to classify them precisely. From an overall perspective, our method can perform very well on these two open datasets. In the SHU-ET dataset, e-telegram, the class with fewest samples, is only classified with 79% accuracies. Therefore, the performance on this dataset is diminished. In a word, for ETC problems, dataset imbalance exists ubiquitously and should be taken into consideration. Sample size has positive correlation to model's accuracies, and as a result, separately analyzing unweighted accuracies is less valuable.

E. Comparison With Other Methods

We compare the proposed method with some state-of-the-art ETC methods, including both traffic images and traffic text based methods. Besides, self-supervised tabular classification methods are also used to evaluate the effectiveness of our strategies. The brief introduction of these methods are as follows:

- 1) *IDCNN* [7] is the first one to propose constructing traffic images for the traffic classification problem and outperforms other CNN models such as 2dCNN. Its input is traffic images with size of 28*28, which is constructed by cutting or padding packets to fix the uniform size.
- 2) *ET-bert* [15] treats traffic data as a special language and uses the popular BERT [29] to solve ETC problems. The most important contribution is a special Datagram to Token method used for traffic tokenization. Its input is traffic text where both sequence length and embedding dimension are 784.
- 3) *MT-FlowFormer* [13] proposes an efficient classifier with an attention mechanism to extract features from flow sequences while maintaining low computational costs. The framework adopts a Mean-Teacher style semi-supervised approach and a spatiotemporal data augmentation method to leverage unlabeled traffic data, which explores spatial and temporal relationships within traffic data.
- 4) *NetMamba* [30] is an efficient network traffic classification model using a linear-time unidirectional Mamba architecture for enhanced efficiency and accuracy. Its



crucial byte-level info, reducing biases. Its input is traffic images, which are constructed by 10 flows and 84 features.

5) *Subtab* [28] is a self-supervised learning method for tabular data, which is the first to use subsets of input

TABLE VII
MODELS' PERFORMANCE ON DIFFERENT DATASETS TRAINED WITH VARIOUS PROPORTION OF LABELED SAMPLES

Dataset	Labeled Ratio	Metrics	1DCNN	ET-BERT	MT-FlowFormer	NetMamba	Subtab	Our Method
ISCX VPN-nonVPN	10%	AC	96.12 $\pm$ 0.27	71.00 $\pm$ 0.28	97.31 $\pm$ 0.16	72.70 $\pm$ 0.81	99.22 $\pm$ 0.07	99.30 $\pm$ 0.06
		FS	70.33 $\pm$ 0.66	70.90 $\pm$ 0.23	77.30 $\pm$ 0.77	72.86 $\pm$ 0.03	94.94 $\pm$ 0.76	95.27 $\pm$ 0.49
		RC	67.90 $\pm$ 1.33	70.18 $\pm$ 0.03	75.41 $\pm$ 0.61	72.70 $\pm$ 0.54	94.58 $\pm$ 1.06	94.83 $\pm$ 0.69
		PR	81.20 $\pm$ 8.60	72.50 $\pm$ 7.23	85.20 $\pm$ 0.74	72.22 $\pm$ 0.57	95.98 $\pm$ 0.56	96.32 $\pm$ 0.42
	30%	AC	98.51 $\pm$ 0.19	85.07 $\pm$ 0.70	98.13 $\pm$ 0.12	86.65 $\pm$ 0.04	99.50 $\pm$ 0.06	99.56 $\pm$ 0.04
		FS	89.63 $\pm$ 1.80	84.79 $\pm$ 0.72	80.10 $\pm$ 0.91	86.48 $\pm$ 0.52	96.92 $\pm$ 0.47	97.24 $\pm$ 0.41
		RC	86.96 $\pm$ 1.71	93.98 $\pm$ 0.88	79.08 $\pm$ 0.14	86.65 $\pm$ 0.74	96.78 $\pm$ 0.57	97.03 $\pm$ 0.46
		PR	93.73 $\pm$ 1.46	85.92 $\pm$ 0.63	87.12 $\pm$ 0.95	86.33 $\pm$ 0.66	97.16 $\pm$ 0.49	97.57 $\pm$ 0.50
	50%	AC	98.92 $\pm$ 0.10	89.47 $\pm$ 0.39	98.44 $\pm$ 0.06	90.70 $\pm$ 0.63	99.60 $\pm$ 0.05	99.65 $\pm$ 0.03
		FS	93.29 $\pm$ 0.50	89.41 $\pm$ 0.38	84.20 $\pm$ 0.78	90.69 $\pm$ 0.15	97.55 $\pm$ 0.51	97.85 $\pm$ 0.34
		RC	91.40 $\pm$ 0.73	88.77 $\pm$ 0.35	83.90 $\pm$ 0.76	90.70 $\pm$ 0.53	97.37 $\pm$ 0.60	97.62 $\pm$ 0.43
		PR	95.73 $\pm$ 0.44	90.22 $\pm$ 0.71	89.20 $\pm$ 0.34	90.45 $\pm$ 0.40	97.82 $\pm$ 0.43	98.20 $\pm$ 0.31
	70%	AC	99.11 $\pm$ 0.04	93.27 $\pm$ 0.05	98.82 $\pm$ 0.09	92.62 $\pm$ 0.07	99.65 $\pm$ 0.04	99.69 $\pm$ 0.03
		FS	93.96 $\pm$ 0.26	93.26 $\pm$ 0.16	88.32 $\pm$ 0.18	92.74 $\pm$ 0.90	97.78 $\pm$ 0.36	98.11 $\pm$ 0.25
		RC	92.81 $\pm$ 0.59	92.67 $\pm$ 0.26	88.13 $\pm$ 0.34	92.62 $\pm$ 0.36	97.58 $\pm$ 0.55	97.84 $\pm$ 0.41
		PR	95.41 $\pm$ 0.22	93.94 $\pm$ 0.30	92.43 $\pm$ 0.80	92.51 $\pm$ 0.03	98.08 $\pm$ 0.23	98.44 $\pm$ 0.20
	100%	AC	99.41 $\pm$ 0.04	95.39 $\pm$ 0.34	99.03 $\pm$ 0.08	92.42 $\pm$ 0.84	99.70 $\pm$ 0.04	99.73 $\pm$ 0.04
		FS	96.29 $\pm$ 0.20	95.32 $\pm$ 0.10	92.13 $\pm$ 0.78	92.41 $\pm$ 0.77	98.05 $\pm$ 0.32	98.31 $\pm$ 0.31
		RC	95.71 $\pm$ 0.60	95.18 $\pm$ 0.17	91.08 $\pm$ 0.91	92.42 $\pm$ 0.37	97.84 $\pm$ 0.50	98.16 $\pm$ 0.37
		PR	96.96 $\pm$ 0.30	95.82 $\pm$ 0.38	93.02 $\pm$ 0.29	92.27 $\pm$ 0.52	98.32 $\pm$ 0.20	98.52 $\pm$ 0.33
ISCX Tor-nonTor	10%	AC	95.96 $\pm$ 0.30	69.63 $\pm$ 1.57	93.11 $\pm$ 0.88	91.51 $\pm$ 0.76	99.51 $\pm$ 0.03	99.47 $\pm$ 0.09
		FS	76.71 $\pm$ 0.60	40.32 $\pm$ 7.02	70.22 $\pm$ 0.55	90.91 $\pm$ 0.10	96.11 $\pm$ 1.04	96.81 $\pm$ 1.54
		RC	75.90 $\pm$ 0.73	41.61 $\pm$ 5.89	70.91 $\pm$ 0.35	91.51 $\pm$ 0.54	95.33 $\pm$ 1.09	95.93 $\pm$ 1.68
		PR	78.32 $\pm$ 0.10	48.85 $\pm$ 9.95	71.03 $\pm$ 0.21	90.74 $\pm$ 0.38	98.01 $\pm$ 1.48	98.36 $\pm$ 1.38
	30%	AC	98.99 $\pm$ 0.12	83.87 $\pm$ 0.61	96.59 $\pm$ 0.88	92.98 $\pm$ 0.57	99.71 $\pm$ 0.02	99.69 $\pm$ 0.07
		FS	90.14 $\pm$ 1.88	64.59 $\pm$ 5.68	80.03 $\pm$ 0.72	92.57 $\pm$ 0.19	98.09 $\pm$ 0.42	98.29 $\pm$ 0.44
		RC	88.77 $\pm$ 1.52	63.99 $\pm$ 5.62	79.35 $\pm$ 0.20	92.98 $\pm$ 0.22	97.62 $\pm$ 0.87	98.09 $\pm$ 0.79
		PR	93.09 $\pm$ 2.97	67.84 $\pm$ 6.09	83.69 $\pm$ 0.15	92.72 $\pm$ 0.43	98.98 $\pm$ 0.41	98.77 $\pm$ 0.48
	50%	AC	99.27 $\pm$ 0.11	91.20 $\pm$ 0.38	97.29 $\pm$ 0.20	94.22 $\pm$ 0.54	99.75 $\pm$ 0.04	99.76 $\pm$ 0.05
		FS	91.37 $\pm$ 2.02	78.69 $\pm$ 2.28	90.39 $\pm$ 0.59	93.94 $\pm$ 0.92	98.74 $\pm$ 0.58	98.88 $\pm$ 0.43
		RC	90.31 $\pm$ 1.91	79.38 $\pm$ 3.19	89.39 $\pm$ 0.46	94.22 $\pm$ 0.29	98.54 $\pm$ 0.82	98.79 $\pm$ 0.62
		PR	95.67 $\pm$ 2.47	81.27 $\pm$ 0.57	92.91 $\pm$ 0.18	93.96 $\pm$ 0.22	99.22 $\pm$ 0.36	99.05 $\pm$ 0.45
	70%	AC	99.48 $\pm$ 0.04	94.55 $\pm$ 0.20	97.77 $\pm$ 0.89	95.23 $\pm$ 0.56	99.77 $\pm$ 0.03	99.78 $\pm$ 0.04
		FS	95.63 $\pm$ 0.94	88.76 $\pm$ 3.15	93.53 $\pm$ 0.25	95.03 $\pm$ 0.98	99.18 $\pm$ 0.33	99.16 $\pm$ 0.35
		RC	94.33 $\pm$ 0.96	89.31 $\pm$ 2.46	92.83 $\pm$ 0.28	95.23 $\pm$ 0.50	99.07 $\pm$ 0.35	99.18 $\pm$ 0.22
		PR	98.65 $\pm$ 0.79	89.23 $\pm$ 3.46	95.12 $\pm$ 0.15	95.02 $\pm$ 0.63	99.30 $\pm$ 0.38	99.18 $\pm$ 0.48
	100%	AC	99.61 $\pm$ 0.05	96.94 $\pm$ 0.37	98.81 $\pm$ 1.03	96.22 $\pm$ 0.56	99.80 $\pm$ 0.02	99.81 $\pm$ 0.03
		FS	96.31 $\pm$ 0.43	92.68 $\pm$ 1.49	94.55 $\pm$ 0.05	96.09 $\pm$ 0.37	99.24 $\pm$ 0.17	99.25 $\pm$ 0.17
		RC	95.28 $\pm$ 0.72	92.88 $\pm$ 0.66	93.13 $\pm$ 0.59	96.22 $\pm$ 0.56	99.10 $\pm$ 0.35	99.22 $\pm$ 0.23
		PR	98.35 $\pm$ 0.62	92.88 $\pm$ 2.13	97.00 $\pm$ 0.09	96.11 $\pm$ 0.88	99.38 $\pm$ 0.02	99.34 $\pm$ 0.31
SHU-ET	10%	AC	73.03 $\pm$ 2.92	84.84 $\pm$ 0.60	78.06 $\pm$ 0.26	97.81 $\pm$ 0.32	96.45 $\pm$ 0.12	97.16 $\pm$ 0.17
		FS	59.77 $\pm$ 4.84	68.26 $\pm$ 1.61	59.49 $\pm$ 0.09	97.69 $\pm$ 0.26	92.56 $\pm$ 0.68	93.57 $\pm$ 0.68
		RC	58.96 $\pm$ 3.65	69.26 $\pm$ 1.43	55.34 $\pm$ 0.10	97.81 $\pm$ 0.75	92.05 $\pm$ 0.75	92.99 $\pm$ 0.74
		PR	68.99 $\pm$ 8.13	68.08 $\pm$ 2.31	64.89 $\pm$ 0.09	97.71 $\pm$ 0.96	93.59 $\pm$ 1.05	94.54 $\pm$ 0.96
	30%	AC	90.82 $\pm$ 1.87	91.91 $\pm$ 0.57	93.51 $\pm$ 0.21	97.97 $\pm$ 0.65	97.42 $\pm$ 0.20	98.23 $\pm$ 0.10
		FS	82.40 $\pm$ 1.47	79.99 $\pm$ 4.01	72.07 $\pm$ 0.09	97.94 $\pm$ 0.81	94.30 $\pm$ 0.71	95.70 $\pm$ 0.57
		RC	81.19 $\pm$ 1.65	78.97 $\pm$ 3.77	70.35 $\pm$ 0.15	97.97 $\pm$ 0.05	93.99 $\pm$ 0.58	95.38 $\pm$ 0.86
		PR	84.36 $\pm$ 1.00	84.01 $\pm$ 5.79	73.98 $\pm$ 0.20	97.93 $\pm$ 0.61	94.85 $\pm$ 0.90	96.26 $\pm$ 0.45
	50%	AC	95.39 $\pm$ 0.20	93.46 $\pm$ 0.34	92.52 $\pm$ 0.21	98.32 $\pm$ 0.51	97.65 $\pm$ 0.21	98.54 $\pm$ 0.10
		FS	87.17 $\pm$ 0.21	82.83 $\pm$ 1.23	76.10 $\pm$ 0.05	98.28 $\pm$ 0.07	94.87 $\pm$ 0.61	96.42 $\pm$ 0.38
		RC	86.84 $\pm$ 0.27	83.35 $\pm$ 1.91	76.30 $\pm$ 0.10	98.32 $\pm$ 0.47	94.59 $\pm$ 0.58	96.21 $\pm$ 0.48
		PR	90.26 $\pm$ 3.67	83.56 $\pm$ 0.43	76.37 $\pm$ 0.09	98.29 $\pm$ 0.51	95.32 $\pm$ 0.87	96.90 $\pm$ 0.32
	70%	AC	95.25 $\pm$ 0.56	94.48 $\pm$ 0.56	96.75 $\pm$ 0.19	98.63 $\pm$ 0.24	97.78 $\pm$ 0.16	98.71 $\pm$ 0.10
		FS	87.17 $\pm$ 0.55	83.78 $\pm$ 1.12	88.11 $\pm$ 0.09	98.64 $\pm$ 0.09	95.05 $\pm$ 0.46	96.82 $\pm$ 0.26
		RC	86.69 $\pm$ 0.85	82.86 $\pm$ 1.37	88.29 $\pm$ 0.06	98.63 $\pm$ 0.66	94.79 $\pm$ 0.31	96.72 $\pm$ 0.45
		PR	90.14 $\pm$ 3.66	85.43 $\pm$ 1.55	87.98 $\pm$ 0.12	98.62 $\pm$ 0.47	95.47 $\pm$ 0.70	97.17 $\pm$ 0.23
	100%	AC	96.24 $\pm$ 0.53	95.63 $\pm$ 0.29	97.24 $\pm$ 0.27	98.97 $\pm$ 0.41	97.86 $\pm$ 0.19	98.83 $\pm$ 0.08
		FS	88.70 $\pm$ 0.31	88.19 $\pm$ 1.24	88.94 $\pm$ 0.22	98.97 $\pm$ 0.06	95.23 $\pm$ 0.63	97.12 $\pm$ 0.10
		RC	88.07 $\pm$ 0.65	87.95 $\pm$ 2.16	89.37 $\pm$ 0.22	98.97 $\pm$ 0.41	94.96 $\pm$ 0.48	96.95 $\pm$ 0.20
		PR	92.46 $\pm$ 0.99	89.36 $\pm$ 0.51	88.61 $\pm$ 0.20	98.96 $\pm$ 0.45	95.67 $\pm$ 0.83	97.45 $\pm$ 0.21

The **bold** font means the best result and the **blue** font represents the second one.
AC, FS, RC and PR represent Accuracy, F1-score, Recall and Precision respectively.

tables to improve performance. Like our method, it first divides the input tables into sub-tables. However, for each sub-table, it uses an encoder-decoder to reconstruct the original input. Besides, it can choose whether calculating contrastive loss and distance loss between all subsets instead of different samples.

For 1DCNN, ET-bert, MT-FlowFormer and NetMamba, data processing methods are strictly followed by corresponding papers and code. Thus, their input datatypes are not in tabular format. Subtab is a self-supervised method for tabular data, so it uses the same processed traffic tables as our method. To test the model's sensitivity on labeled samples, which can

TABLE VIII

THE DIFFERENCES BETWEEN SUBTAB AND OUR METHOD ON PARAMETERS, THE NUMBER OF COMPUTATIONS AND PROCESSING TIME

Metrics	Subtab	Our method
Parameters	0.5M	0.5M
The Number of Reconstruction Loss Computations	7	2
The Number of Contrastive Loss Computations	21	1
The Number of Distance Loss Computations	21	1
Processing Time (s/item)	0.55	0.27
Processing Speed (items/s)	1.83	3.76

The **bold** font represents the best result.

also be viewed as the dependence on labeled data, the proportion of labeled samples used for training will be decreased. Specifically, for each dataset, we train all models with 10%, 30%, 50%, 70% and 100% labeled samples, respectively. It is important to emphasize that the size of test sets is maintained the same during this process, which is at 20% of the total dataset. Besides, we repeat the train-test split 10 times to collect average values and standard deviations. Results are displayed in Table VII.

As mentioned before, traffic datasets suffer great imbalance problem. Therefore, the unweighted accuracies are consistently inflated, resulting in a certain degree of distortion. As a consequence, it is better to analyze other weighted metrics. From the table, it can be found that our method ranks the top two in most metrics across all the conditions. Specifically, this method achieves the best results (bold font) in ISCX VPN-nonVPN and ISCX Tor-nonTor datasets. On ISCX VPN-nonVPN dataset, by only using 10% labeled data, the model can obtain average 99.30% in Accuracy, 95.27% in F1-score, 94.83% in Recall and 96.32% in Precision, while on ISCX Tor-nonTor dataset, these metrics are 99.47%, 96.81%, 95.93% and 98.36%. These results are within 4% of those obtained from training with 100% labeled samples, and this indicates that at pretraining stage, the encoder has effectively learned latent representations of input traffic tables and achieved satisfactory clustering results. As a result, at downstream stage, even a small number of labeled samples is enough to train a great classification network. It can be found that most methods perform better on ISCX Tor-nonTor dataset, thus it can be concluded that it is easier for methods to classify Tor data than VPN data. Moreover, Subtab, which is also a traffic-table based method, ranks second. These two methods significantly surpass other traffic-images and traffic-text based methods. Therefore, this demonstrates the feasibility and rationality of utilizing traffic tables to address ETC problem.

On SHU-ET dataset, however, NetMamba gets the best result while our method ranks second. As mentioned in Section III, the model should to classify applications while the size of SHU-ET is also smaller. From the dataset perspective, smaller size means fewer packets, and after data processing, the number of traffic tables will be further compressed since one table needs 14 packets. In contrast, only 5 packets are required for one image in NetMamba. As a result, samples used for training are relatively more plentiful in NetMamba than our method. Furthermore, we analyzed the number of parameters between these two methods. NetMamba has about 110M learnable parameters, in contrast to our method, which has only 0.5M. With more parameters and samples, NetMamba can better learn distinct features. Even so, the difference

in classification performance between the two methods is within 5%. Another question should be answered is that why NetMamba's performance gets a great decline in ISCX VPN-nonVPN and ISCX Tor-nonTor datasets. The possible reason is that NetMamba is not designed to deal with the imbalanced datasets. Compared to SHU-ET, two ISCX datasets suffer severer imbalance problem, which has been shown in Fig. 6. During the implementation of the open source code, at first it will automatically delete the classes with a small number of samples. For example, on ISCX VPN-nonVPN dataset, it only keeps seven classes. After forcing the model not to delete any classes, it comes to current results. However, the SHU-ET dataset has not only a smaller size, but also a slight imbalance problem. Therefore, the influence is not as prominent as the other two datasets.

Although the classification performance is improved by our method, this improvement is not very huge compared to Subtab. Subtab deals with each subset more carefully, which can be viewed from a local perspective, while our method takes a global perspective on traffic tables. Because of this, Subtab ignores other features of traffic tables and cannot do multi-feature fusion, which limits the performance. Compared to classification performance, our method gets a more significant improvement in the aspect of processing speed. Table VIII shows differences between two methods in parameters, the number of computations, processing time and processing speed. Such a significant improvement in speed is attributed to the reduction in the number of computations. In Subtab, for one traffic table, each subset is used to reconstruct it once, so the reconstruction loss is computed 7 times. What's more, the contrastive and the distance losses are computed by pairs of subsets, thus for 7 subsets, they are computed $C_7^2 = 21$ times. Since our method only use two aggregated version of subsets to do reconstruction and comparison, it only needs to compute the reconstruction loss twice and other two losses once for each traffic table. By this way, the average processing speed increases by nearly two times, from 1.83 items per second to 3.76 items per second.

In summary, three conclusions can be drawn from experimental results. Firstly, traffic tables are suitable for addressing ETC problems. Secondly, our method can achieve great performance on both classifying services and applications. Meanwhile, our method can maintain a great performance when facing with imbalance dataset. Finally, compared to original Subtab, our method can do multi-feature fusion to obtain better results and improve the processing time by reducing the number of computations.

VI. ABLATION STUDY

In this section, we perform an ablation study to better understand why our method can achieve great performance. All the experiments are conducted on the ISCX VPN-nonVPN dataset, which is elaborated in Section III.

A. Loss Function Weights

As mentioned before, the loss function is made up of the reconstruction loss l_r , the contrastive loss l_c and the distance loss l_d , as shown in Equation (7).

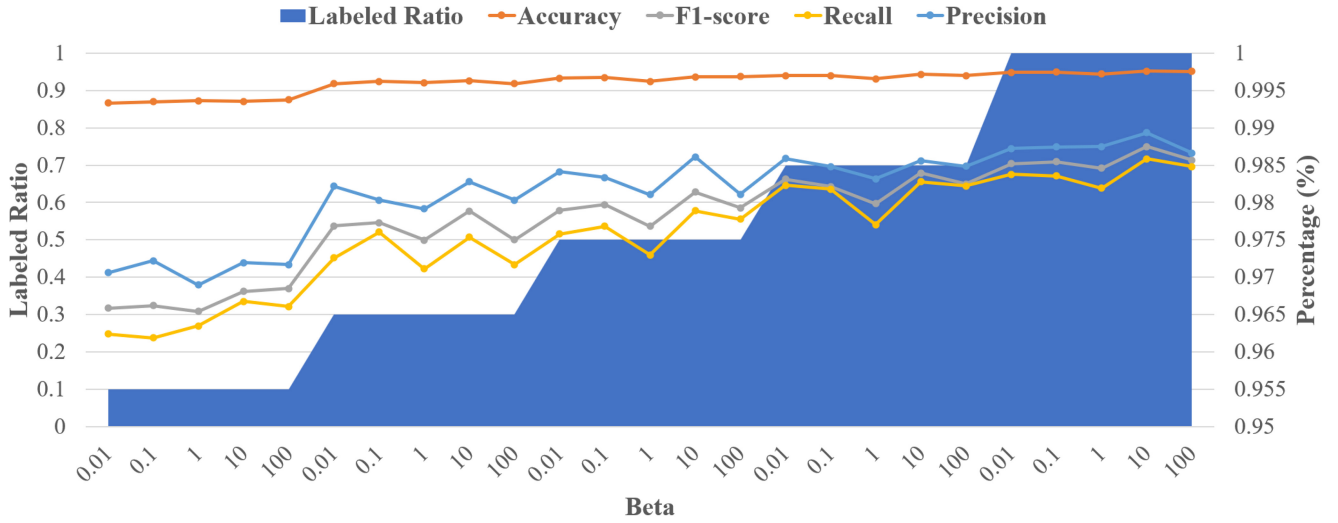


Fig. 7. The trend of model's performance and β under different labeled data ratios. The left coordinate represents the labeled data ratio used to training the downstream classification network, and each blue trapezoid represents a specific labeled ratio. For each labeled ratio, β is increased from 0.01 to 100 with a step of 10, and the right coordinate represents the percentages of evaluation metrics.

Furthermore, the total reconstruction loss l_r is a weighted sum of two components: the partial subsets reconstruction loss l_{ro} and the corrupted subsets reconstruction loss l_{rc} , as shown in Equation (4). Given the distinct roles of the loss functions, the various weight combinations among them will determine the model's focus on different aspects of the task. Thus, experiments are conducted to find the optimal combination of α , β and γ .

Regarding the roles of the three loss functions, l_r and l_d only serve to pull positive pairs closer, whereas l_c serves to pull positive pairs closer and push negative pairs apart. Therefore, the weight of the contrastive loss needs to be first taken into consideration and we freeze α and γ to find the best β . Specifically, α and β are fixed to 0.7 and 1, respectively. The initial value of β is set to 0.01 and is gradually increased with a step size of 10. Fig. 7 illustrates the trend between β and the model's performance. Since traffic dataset often suffers severe imbalanced problem as mentioned before, we still focus on another three weighted metrics, which are F1-score, Recall and Precision. From Fig. 7, the relationship between β and the performance can be found. Under a specific condition (a certain labeled ratio), as β increases, the majority of evaluation metrics undergo a process of initial decrease, followed by an increase, and then a subsequent decline. With a small β , the total loss function will equal to l_r , and lower l_r means better reconstruction effect. Since l_c is computed by reconstructed input, lower l_r can also benefit l_c . As a result, when β equals 0.01, the performance is great. However, as β is increased, this benefit will be diminished. When l_r and l_c are of the same order of magnitude (in our experiments, β equals 10), during the backpropagation process, both can be optimized to the maximum extent simultaneously. Thus, another improvement is obtained. Nevertheless, if l_c surpasses l_r too much, the performance will decrease again since it's hard for l_r to benefit from l_c . Considering all circumstances, we select 10 as the optimal value of β . Additionally, we recommend to set a small value for l_c at

the beginning. By observing the difference between l_r and l_c , we can adjust β to ensure that both l_c and l_r are of similar magnitude.

Then, we freeze β and γ to find the best α . Fig. 8 shows the trend of α and the model's performance. With a small α , the corrupted subsets reconstruction loss will account for a bigger part of the total l_r . Since corrupted subsets reconstruction task is much more difficult than just partial subsets reconstruction, α reflects the difficulty of the pretext task. Both too difficult pretext task or too easy pretext task will weaken the performance. With big α , the reconstruction task is too hard for model to learn the useful representations, while small α is not enough to learn the best representations. From the results, we choose a moderate 0.3 as the optimal α . Besides, we recommend to set this value at the first time of training.

Since l_d has the same function as l_c and l_r , it is not as important as previous ones. It is recommended to be 1 in the experiment.

B. RSS and Temporal Feature Extraction

Since the RSS algorithm is an important strategy to solve the lack of augmentation methods for tabular data, its effectiveness is carefully researched. As mentioned before, two latent representations will be used for calculating the loss functions. One is aggregated from subsets without corruption, and the other is aggregated from corrupted subsets, which can be denoted as l and l' , respectively. Four different scenarios are being designed to find the best strategy. The first scenario is using all subsets and their corrupted version to aggregate two latent representations, denoted as Deterministic Certainty (DC). In the second scenario, the previous four subsets from x_1 to x_4 are used to aggregate l in order, while the remaining three subsets corrupted by adding noise are used to aggregate l' . We denote this scenario as Most Certainty (MC). In the third scenario, we still choose four subsets to aggregate l , but they

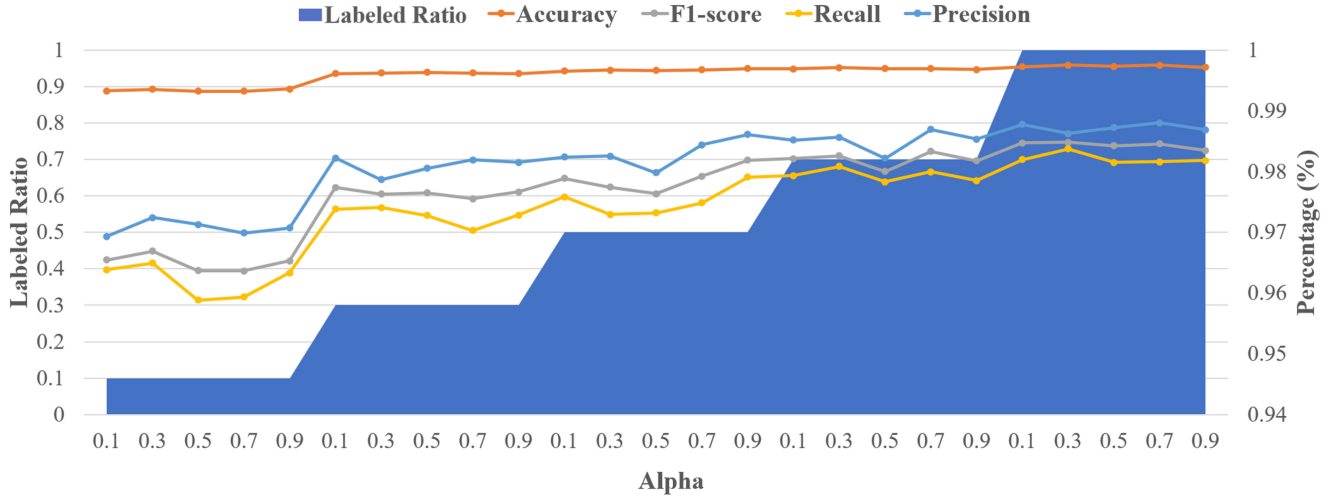


Fig. 8. The trend of model's performance and α under different labeled data ratios. The left coordinate represents the labeled data ratio used to training the downstream classification network, and each blue trapezoid represents a specific labeled ratio. For each labeled ratio, β is increased from 0.1 to 0.9 with a step of 0.1, and the right coordinate represents the percentages of evaluation metrics.

TABLE IX
THE RELATIONSHIP BETWEEN DIFFERENT STRATEGIES AND THE MODEL'S PERFORMANCE

Scenario	Subsets Count	Subsets Location	Use Time Feature	Labeled Ratio	Accuracy (%)	F1-score (%)	Recall (%)	Precision (%)
DC+TF	All	In order	Yes	10%	99.317 $\pm$ 0.051	95.779 $\pm$ 0.325	95.384 $\pm$ 0.525	96.448 $\pm$ 0.518
				30%	99.586 $\pm$ 0.030	97.546 $\pm$ 0.232	97.331 $\pm$ 0.240	97.862 $\pm$ 0.286
				50%	99.660 $\pm$ 0.027	97.964 $\pm$ 0.206	97.798 $\pm$ 0.153	98.204 $\pm$ 0.345
				70%	99.692 $\pm$ 0.020	98.211 $\pm$ 0.258	98.047 $\pm$ 0.215	98.462 $\pm$ 0.358
				100%	99.735 $\pm$ 0.025	98.556 $\pm$ 0.206	98.334 $\pm$ 0.251	98.814 $\pm$ 0.178
MC+TF	4 and 3	In order	Yes	10%	96.663 $\pm$ 0.140	86.947 $\pm$ 0.832	85.114 $\pm$ 0.818	90.627 $\pm$ 0.258
				30%	97.336 $\pm$ 0.103	90.651 $\pm$ 0.313	89.524 $\pm$ 0.545	93.229 $\pm$ 0.385
				50%	97.573 $\pm$ 0.057	91.844 $\pm$ 0.337	90.911 $\pm$ 0.333	93.549 $\pm$ 0.604
				70%	97.642 $\pm$ 0.111	92.156 $\pm$ 0.180	91.258 $\pm$ 0.501	94.068 $\pm$ 0.319
				100%	97.730 $\pm$ 0.095	92.784 $\pm$ 0.187	91.870 $\pm$ 0.388	94.383 $\pm$ 0.370
FC+TF	4 and 3	Random	Yes	10%	99.391 $\pm$ 0.045	96.534 $\pm$ 0.285	96.258 $\pm$ 0.217	97.048 $\pm$ 0.365
				30%	99.615 $\pm$ 0.031	97.688 $\pm$ 0.311	97.520 $\pm$ 0.342	97.916 $\pm$ 0.265
				50%	99.688 $\pm$ 0.025	98.149 $\pm$ 0.253	97.967 $\pm$ 0.328	98.375 $\pm$ 0.261
				70%	99.733 $\pm$ 0.016	98.461 $\pm$ 0.184	98.296 $\pm$ 0.212	98.681 $\pm$ 0.174
				100%	99.761 $\pm$ 0.016	98.634 $\pm$ 0.154	98.497 $\pm$ 0.110	98.826 $\pm$ 0.258
NC+TF	Random	Random	Yes	10%	99.401 $\pm$ 0.059	96.535 $\pm$ 0.397	96.319 $\pm$ 0.395	96.887 $\pm$ 0.468
				30%	99.645 $\pm$ 0.022	97.939 $\pm$ 0.253	97.757 $\pm$ 0.179	98.170 $\pm$ 0.399
				50%	99.706 $\pm$ 0.031	98.304 $\pm$ 0.276	98.168 $\pm$ 0.200	98.503 $\pm$ 0.387
				70%	99.739 $\pm$ 0.023	98.515 $\pm$ 0.222	98.465 $\pm$ 0.090	98.600 $\pm$ 0.385
				100%	99.769 $\pm$ 0.022	98.692 $\pm$ 0.186	98.629 $\pm$ 0.109	98.781 $\pm$ 0.311
NC	Random	Random	No	10%	99.400 $\pm$ 0.038	96.475 $\pm$ 0.181	96.278 $\pm$ 0.201	96.893 $\pm$ 0.329
				30%	99.631 $\pm$ 0.027	97.785 $\pm$ 0.167	97.628 $\pm$ 0.261	98.046 $\pm$ 0.230
				50%	99.700 $\pm$ 0.012	98.191 $\pm$ 0.138	97.990 $\pm$ 0.130	98.466 $\pm$ 0.176
				70%	99.732 $\pm$ 0.022	98.376 $\pm$ 0.204	98.288 $\pm$ 0.230	98.574 $\pm$ 0.233
				100%	99.762 $\pm$ 0.016	98.601 $\pm$ 0.118	98.474 $\pm$ 0.119	98.803 $\pm$ 0.121

The **bold** font represents the best result.

will not be in order any longer. Instead, they will be randomly selected from any location. Still, the remaining three subsets will be corrupted and used to aggregate l' . This scenario is denoted as Few Certainty (FC). In the fourth scenario, for each latent representation, the number of subsets as well as their locations are all randomly selected, so we call it No Certainty (NC). What's more, we take the Temporal Feature (TF) into consideration. Table IX shows the influences of these different strategies on model's performance.

Results show that the NC+TF strategy get the best results in most circumstances. Each subset can be viewed as a

perspective of one traffic table, and different combinations of subsets represent various scales of the perspectives. Since both subsets and their combinations are random in the NC strategy, it gives model multiscale and multi-views of a certain traffic table. Notably, MC strategy goes through a huge decrease in performance. It shows a negative example that a fixed but not full scaled perspective of input will force model to learn the similarity of these two perspectives, which changes the purpose of task. What's more, another key point is that two parts are not allowed to have duplicate subsets. If there have duplicate subsets, these subsets will be given bigger weights

during the backpropagation and decrease the performance. In summary, RSS algorithm helps the model to learn the most basic and informative features and to avoid overfitting problem, like dropout.

Since MT-TF performs better than MT, it can be found that temporal features also exist in the traffic tables. Even a simple extraction method can further improve the performance. In a word, these results prove that RSS strategy can be an effective data augmentation method for tabular data, and improvements of performance can be obtained by fusion with temporal features.

VII. CONCLUSION

In this work, a multi-feature fusion based self-supervised learning approach for traffic tables is proposed and implemented. A reconstruction task and contrastive task are designed to learn better latent representations of input traffic tables. The contrastive task depends on data augmentation methods, and the RSS algorithm is proposed to solve the problem that traffic tables lack data augmentation methods. It randomly chooses some subsets to add noise while keeping the other subsets still. After that, the latent representations are aggregated by two parts of subsets, respectively. An attention-pooling module is used to learn the weights of subsets, generating better latent representations. What's more temporal features are extracted from the reconstruction inputs and fused with the latent representations. Experimental results on two open datasets and one self-collected dataset have shown that traffic tables can effectively solve ETC problems even with a small number of labeled data on imbalanced datasets. Compared to the state-of-the-art tabular self-supervised learning method, classification performance is improved on all three datasets. Furthermore, greater improvement is achieved on processing speed, which increases by almost two times, from 1.83 tables per second to 3.76 tables per second. As a part of our future work, we will develop a better method to extract diverse features from traffic tables and put our method into practical applications.

REFERENCES

- [1] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
- [2] A. Bremler-Barr, Y. Harchol, D. Hay, and Y. Koral, "Deep packet inspection as a service," in *Proc. 10th ACM Int. Conf. Emerg. Netw. Exp. Technol.*, 2014, pp. 271–282.
- [3] X. Zheng, X. Ma, Y. Jin, D. Gu, and R. Wang, "Tabular-based self-supervised learning approach for encrypted traffic classification," *J. Electron. Imag.*, vol. 32, no. 4, 2023, Art. no. 43032.
- [4] T. Auld, A. W. Moore, and S. F. Gull, "Bayesian neural networks for Internet traffic classification," *IEEE Trans. Neural Netw.*, vol. 18, no. 1, pp. 223–239, Jan. 2007.
- [5] A. W. Moore and K. Papagiannaki, "Toward the accurate identification of network applications," in *Proc. 6th Int. Workshop, Passive Act. Netw. Meas.*, 2005, pp. 41–54.
- [6] M. Shen, Y. Liu, L. Zhu, X. Du, and J. Hu, "Fine-grained webpage fingerprinting using only packet length information of encrypted traffic," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2046–2059, 2021.
- [7] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Security Inform. (ISI)*, 2017, pp. 43–48.
- [8] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, 2017, pp. 712–717.
- [9] J. Zhang, C. Chen, Y. Xiang, W. Zhou, and A. V. Vasilakos, "An effective network traffic classification method with unknown flow detection," *IEEE Trans. Netw. Service Manag.*, vol. 10, no. 2, pp. 133–147, Jun. 2013.
- [10] K. Lin, X. Xu, and Y. Jiang, "A new semi-supervised approach for network encrypted traffic clustering and classification," in *Proc. IEEE 25th Int. Conf. Comput. Support. Cooperative Work Design (CSCWD)*, 2022, pp. 41–46.
- [11] A. S. Ilyasu and H. Deng, "Semi-supervised encrypted traffic classification with deep convolutional generative adversarial networks," *IEEE Access*, vol. 8, pp. 118–126, 2020.
- [12] O. Aouedi, K. Piamrat, and D. Bagadthey, "A semi-supervised stacked autoencoder approach for network traffic classification," in *Proc. IEEE 28th Int. Conf. Netw. Protocols (ICNP)*, 2020, pp. 1–6.
- [13] R. Zhao, X. Deng, Z. Yan, J. Ma, Z. Xue, and Y. Wang, "MT-FlowFormer: A semi-supervised flow transformer for encrypted traffic classification," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Min.*, 2022, pp. 2576–2584.
- [14] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.
- [15] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.
- [16] H. Y. He, Z. G. Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidosc., Ind.-Driven Digit. Transform. (ITU K)*, 2020, pp. 1–8.
- [17] W. Wei, T. Ju, H. Liao, W. Zhao, and H. Gu, "FLAG: Flow representation generator based on self-supervised learning for encrypted traffic classification," in *Proc. 5th Asia-Pac. Workshop Netw.*, 2021, pp. 14–20.
- [18] Z. Zou, J. Ge, H. Zheng, Y. Wu, C. Han, and Z. Yao, "Encrypted traffic classification with a convolutional long short-term memory neural network," in *Proc. IEEE 20th Int. Conf. High Perform. Comput. Commun.; IEEE 16th Int. Conf. Smart City; IEEE 4th Int. Conf. Data Sci. Syst. (HPCC/SmartCity/DSS)*, 2018, pp. 329–334.
- [19] K. Lin, X. Xu, and H. Gao, "TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT," *Comput. Netw.*, vol. 190, May 2021, Art. no. 107974.
- [20] Y. Zeng, H. Gu, W. Wei, and Y. Guo, "Deep – full – range: A deep learning based network encrypted traffic classification and intrusion detection framework," *IEEE Access*, vol. 7, pp. 45182–45190, 2019.
- [21] J. Yoon, Y. Zhang, J. Jordon, and M. Van der Schaar, "VIME: Extending the success of self-and semi-supervised learning to tabular domain," in *Proc. 34th Adv. Neural Inf. Process. Syst.*, 2020, pp. 11033–11043.
- [22] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.
- [23] Y. Fu, H. Xiong, X. Lu, J. Yang, and C. Chen, "Service usage classification with encrypted Internet traffic in mobile messaging apps," *IEEE Trans. Mobile Comput.*, vol. 15, no. 11, pp. 2851–2864, Nov. 2016.
- [24] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, 2022.
- [25] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
- [26] S.-H. Yoon, J.-W. Park, J.-S. Park, Y.-S. Oh, and M.-S. Kim, "Internet application traffic classification using fixed IP-port," in *Proc. 12th Asia-Pac. Netw. Oper. Manag. Symp. Manag. Enabling Future Internet Changing Bus. New Comput. Services (APNOMS)*, 2009, pp. 21–30.
- [27] D. Bahri, H. Jiang, Y. Tay, and D. Metzler, "SCARF: Self-supervised contrastive learning using random feature corruption," 2022, *arXiv:2106.15147*.
- [28] T. Ucar, E. Hajiramezanali, and L. Edwards, "SubTab: Subsetting features of tabular data for self-supervised representation learning," in *Proc. 35th Adv. Neural Inf. Process. Syst.*, 2021, pp. 18853–18865.
- [29] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2019, *arXiv:1810.04805*.
- [30] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "NetMamba: Efficient network traffic classification via pre-training unidirectional mamba," 2024, *arXiv:2405.11449*.



Xuan Zheng received the B.Eng. and M.E.I. degrees from Shanghai University, China, in 2021 and 2024, respectively. He is currently working on constructing agents to improve network management by combining encrypted traffic classification and LLM together. Besides, he is also trying to find a general Symbolic Regression method for AI in science, and improve the performance of Automatic Speech Recognition to solve real world problems. His research interests include self-supervised learning, encrypted traffic analysis, and large language models.



Yanliang Jin received the B.S. and M.S. degrees in electrical engineering from Xidian University, Xi'an, China, in 1997 and 2000, respectively, and the Ph.D. degree in communication and information system from Shanghai Jiao Tong University, in 2005. He is currently an Associate Professorship with the School of Communication and Information Engineering, Shanghai University. He has published more than 30 journals/conference papers. His research interests include mobile ad hoc networks, wireless sensor networks, wireless multimedia sensor networks, and wireless broadband access.



Xiuli Ma received the Ph.D. degree from Xidian University, in 2007. She is currently an Associate Professorship with the School of Communication and Information Engineering, Shanghai University. Her research interests include big data and intelligent information processing.



Lifu Xu received the B.Eng. degree from Shanghai University, China, in 2021, where he is currently pursuing the master's degree in electronic information. His current research focuses on encrypted traffic analysis and few-shot classification.



Chun Ke received the B.S. degree in electrical engineering and automation from the Wuhan University of Technology, in 2018. He is currently a Project Manager with H3C, Shanghai, China. His research interests include WIFI technology and application, and enterprise networking.